My web-based capstone is a Token Research Explorer that allows a user to browse a list of tokens, search by name or category, select a token to view detailed information, and save structured AI-generated research output for each token. A user can start on the list view, filter tokens using the search input, select a token to navigate to the detail view, toggle checklist items, paste AI research into the side panel, save it, refresh the browser, and see that the saved research persists. At a high level, the App component coordinates shared state, view selection, and side effects. It owns the selected token, search query, saved research output, and the logic for switching between list and detail views. The page or view components represent user experiences, such as the token list view and token detail view, and they render UI based on props without owning global state. Reusable components like TokenList, TokenCard, TokenDetail, and AISummaryBox are responsible for presentation and localized interaction, requesting state changes through callbacks.

Data flows downward from App through props and flows upward through callback functions. Shared state lives only in App, including the selected token ID and the researchByTokenId object, and changes to that state trigger re-renders across both the main content and the AI panel. When a user saves AI output, the state updates in App, which updates both the side panel and any persisted storage behavior. One meaningful useEffect synchronizes the saved research state with localStorage, ensuring persistence across page refreshes. That effect belongs in App because App owns the research state, and placing it elsewhere would fragment responsibility and risk inconsistent persistence. If it were removed, saved research would disappear on refresh and break expected behavior.

I am most confident in the clarity of state ownership and predictable data flow throughout the app. If I had more time, I would revisit how research data is structured and potentially refactor it into a more scalable format to support additional tokens or categories. As I transition into mobile development, I plan to reuse the same core logic for state ownership, view separation, and persistence patterns, adapting them into a mobile navigation structure while keeping a single source of truth for shared data. I used AI this week to validate component boundaries and effect placement, but I chose the final architecture myself after testing the behavior directly in the running application.